

--1. Crear un bloque PL que recorra un cursor con todos los clientes que vivan en España y pinte el nombre del cliente, su teléfono y su ciudad.

```
DECLARE
    CURSOR c_clientes_esp IS
    SELECT nombrecliente, telefono, ciudad
    FROM cliente
    WHERE pais = 'Spain';
BEGIN
    FOR r_cliente IN c_clientes_esp LOOP
        DBMS_OUTPUT.PUT_LINE(r_cliente.nombrecliente || ' - ' || r_cliente.telefono || ' - ' ||
r_cliente.ciudad);
    END LOOP;
END;
/
```

--2. Crear un bloque PL que le pida al usuario un precio para que introduzca por teclado y haga un cursor con todos los productos que valgan (precio venta) menos del precio introducido. Sacar nombre del producto, precio de venta y dimensiones.

```
DECLARE
    vPrecio producto.precioventa%type := &PRECIO_MAXIMO;
    CURSOR c_productos(v_precio product.precioventa%type) IS
    SELECT nombre, precioventa, dimensiones
    FROM producto
    WHERE precioventa < v_precio;
BEGIN
    DBMS_OUTPUT.PUT_LINE('LISTADO DE PRODUCTOS DE MENOS DE ' || vPrecio || '€');
    FOR r_producto IN c_productos(v_precio) LOOP
        DBMS_OUTPUT.PUT_LINE(r_producto.nombre || ' - ' || r_producto.precioventa || ' - ' ||
r_producto.dimensiones);
    END LOOP;
END;
/
```

--3. Escribir un bloque PL que reciba una cadena y visualice el apellido y el código de empleado de todos los empleados cuyo apellido contenga la cadena especificada. Al finalizar se visualizará el número de empleados mostrados.

```
DECLARE
    v_apellido product.apellido1%type := '&INICIALES';
    v_numero NUMBER := 0;
    CURSOR c_empleados(v_apellido product.apellido1%type) IS
    SELECT codepleado, apellido1
    FROM empleado
    WHERE INSTR(apellido1, v_apellido) > 0;
BEGIN
    FOR r_empleado IN c_empleados(v_apellido) LOOP
        DBMS_OUTPUT.PUT_LINE(r_empleado.codepleado || ' - ' || r_empleado.apellido1);
        v_numero := v_numero + 1;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('TOTAL: ' || v_numero);
END;
/
```

--4. Crear un bloque PL que le pida al usuario un código de empleado y recorra los clientes asociados a ese código de empleado. Por cada cliente contará el número de pedidos y en función de estos le aumentará el límite de crédito.

- 1. Si el cliente tiene entre 1 y 5 pedidos: Aumenta el límite de crédito en un 10%
- 2. Si el cliente tiene entre 6 y 10 pedidos: Aumenta el límite de crédito en un 15%
- 3. Si el cliente tiene más de 10 pedidos: Aumenta el límite de crédito en un 20%
- 4. Si el cliente no tiene pedidos, no se actualiza el límite de crédito.

--Queremos ver en la salida el código de cliente, el número de pedidos que tiene y el límite de crédito antes y después de actualizarse.

```
DECLARE
```

```

v_empleado empleado.codempleado%type := &CODIGO_EMPLEADO;
v_nuevolimite cliente.limitecredito%type;
CURSOR c_clientes(v_codemp empleado.codempleado%type) IS
    SELECT c.codcliente, c.limitecredito, count(distinct p.codpedido) AS numped
    FROM cliente c
    LEFT JOIN pedido p ON c.codcliente = p.codcliente
    WHERE codempleadoventas = v_codemp
    GROUP BY c.codcliente, c.limitecredito;
BEGIN
    FOR r_cliente IN c_clientes(v_empleado) LOOP
        CASE
            WHEN r_cliente.numped BETWEEN 1 AND 5 THEN
                v_nuevolimite := r_cliente.limitecredito * 1.1;
            WHEN r_cliente.numped BETWEEN 6 AND 10 THEN
                v_nuevolimite := r_cliente.limitecredito * 1.15;
            WHEN r_cliente.numped > 10 THEN
                v_nuevolimite := r_cliente.limitecredito * 1.2;
            ELSE
                v_nuevolimite := r_cliente.limitecredito;
        END CASE;
        UPDATE cliente SET limitecredito = v_nuevolimite
        WHERE codcliente = r_cliente.codcliente;
        DBMS_OUTPUT.PUT_LINE(r_cliente.codcliente || ' - ' || r_cliente.numped || ' - ' ||
r_cliente.limitecredito || ' (antes) ' || v_nuevolimite || ' (despues)');
    END LOOP;
END;
/

```

--5. Haz un informe que me dé por cada producto el nombre del producto, el nombre de los clientes que han comprado el producto, el total de clientes que han comprado el producto y si no hay pedidos, que indique que no hay pedidos.

```

DECLARE
    v_totalclientes NUMBER(4);
    CURSOR c_productos IS
        SELECT codproducto, nombre
        FROM producto;

    CURSOR c_clientes(v_producto producto.codproducto%type) IS
        SELECT DISTINCT c.nombrecliente
        FROM cliente c
        JOIN pedido p ON c.codcliente = p.codcliente
        JOIN detalle_pedido dp ON p.codpedido = dp.codpedido
        WHERE dp.codproducto = v_producto;
BEGIN
    FOR r_producto IN c_productos LOOP
        DBMS_OUTPUT.PUT_LINE('Producto: ' || r_producto.nombre);
        v_totalclientes := 0;
        FOR r_cliente IN c_clientes(r_producto.codproducto) LOOP
            v_totalclientes := v_totalclientes + 1;
            DBMS_OUTPUT.PUT_LINE('Cliente: ' || r_cliente.nombrecliente);
        END LOOP;
        CASE v_totalclientes
            WHEN 0 THEN DBMS_OUTPUT.PUT_LINE('No hay pedidos');
            ELSE
                DBMS_OUTPUT.PUT_LINE('Total clientes: ' || v_totalclientes);
            END CASE;
    END LOOP;
END;
/

```

--6. Crea en la tabla de productos un campo llamado "Oferta" que admitirá valores (SI/NO) (estaría bien que el valor por defecto sea NO). Haz un script con un cursor que recorra los productos con

stock de más de 100 unidades y para los que no haya pedidos. Para estos pedidos se marcará el campo oferta con SI. Asegúrate que nadie puede modificar los registros del cursor que estás recorriendo.

```
ALTER TABLE producto ADD
(
    Oferta VARCHAR2(2) DEFAULT 'NO' CHECK (Oferta IN ('SI', 'NO'))
);
DECLARE
    CURSOR c_productos IS
        SELECT codproducto
        FROM producto
        WHERE stock > 100
        AND codproducto NOT IN
        (
            SELECT DISTINCT codproducto
            FROM detalle_pedido
        )
        FOR UPDATE;
BEGIN
    FOR r_producto IN c_productos LOOP
        UPDATE producto SET Oferta = 'SI'
        WHERE CURRENT OF c_productos;
    END LOOP;
END;
/
```

--7. Crea una nueva tabla de productos descatalogados (misma estructura que la tabla de productos). Haz un cursor que recorra los productos con stock de menos de 5 unidades y para los que no haya pedidos. Estos productos hay que meterlos en la tabla de productos descatalogados y borrarlos de la tabla de productos.

```
CREATE TABLE descatalogado AS
(
    SELECT * from producto where 1 = 2
);
DECLARE
    CURSOR c_productos IS
        SELECT *
        FROM producto
        WHERE stock < 5
        AND codproducto NOT IN
        (
            SELECT DISTINCT codproducto
            FROM detalle_pedido
        )
        FOR UPDATE;
BEGIN
    FOR r_prod IN c_productos LOOP
        INSERT INTO descatalogado VALUES r_prod;
        DELETE FROM producto WHERE CURRENT OF c_productos;
    END LOOP;
END;
/
```